TrustGuard

Data Quality Pipeline for Retail Data

Project Report

Built using Python, PySpark, SQL, Delta Lake, and Databricks

Field	Details
Domain	Data Engineering / Data Quality / Retail Analytics
Architecture	Medallion Architecture - Raw Layer -> Clean Layer -> Final Layer
Execution Platform	Databricks Free Edition with Serverless Compute
Project Folder Tool	VS Code for documentation, exported notebook, dataset, screenshots, and GitHub submission
Main Dataset	trustguard_retail_transactions.csv

This report explains the complete TrustGuard project in simple words. It also includes Databricks screenshots that show the pipeline output at each important step.

1. Project Overview

TrustGuard is an end-to-end data quality pipeline built for retail transaction data. The project takes raw and inconsistent data, checks it for common data quality issues, cleans it, separates invalid records, and creates final tables that can be used for business reporting and analysis.

The actual pipeline was executed on Databricks using PySpark, SQL, and Delta Lake. VS Code was used to maintain the final project folder, documentation, screenshots, exported notebook, dataset, and GitHub submission files.

2. Business Problem

Retail data often comes from different systems such as stores, online orders, customer platforms, payment systems, and delivery systems. Because of this, the data may contain missing values, duplicate records, inconsistent date formats, wrong total amounts, and different spellings for the same values.

If this data is used directly for reporting, revenue numbers, customer analysis, and product sales reports can become incorrect. TrustGuard solves this by applying validation, cleaning, rejected record handling, and reporting in a structured pipeline.

3. Architecture Used

Layer	Purpose	Main Tables
Raw Layer	Stores original data with metadata. No cleaning is applied here.	raw_transactions
Clean Layer	Applies validation, standardization, deduplication, and separates invalid records.	clean_transactions, clean_customers, rejected_records, dq_report
Final Layer	Stores analysis-ready tables and reports for business use.	final_transactions, customer_summary, city_sales_report, payment_method_report, product_category_report, anomaly_log, pipeline_log

Simple flow: Raw CSV Dataset -> Raw Delta Table -> Data Quality Checks -> Cleaned Data + Rejected Records -> Final Tables -> SQL Reports and Logs.

4. Data Quality Checks and Cleaning Rules

Check / Rule	What it does
Completeness Check	Finds missing values in important fields like transaction_id, customer_id, product_category, payment_method, quantity, unit_price, total_amount, and transaction_date.
Duplicate Check	Finds repeated transaction_id values and keeps only one valid record in the clean table.
Date Standardization	Converts mixed date formats into a consistent date format wherever possible.
Text Standardization	Standardizes values like payment method, gender, city, and active status.
Range Check	Checks that quantity and price values are valid and greater than zero.
Amount Validation	Checks whether total_amount matches quantity multiplied by unit_price.
Rejected Records	Stores records that fail critical checks with a clear rejection reason.
Anomaly Detection	Flags unusual transactions such as very high quantity or very high transaction amount.

The pipeline does not silently delete invalid records. Instead, it sends them to the rejected_records table with a reason. This makes the pipeline easier to audit, explain, and debug.

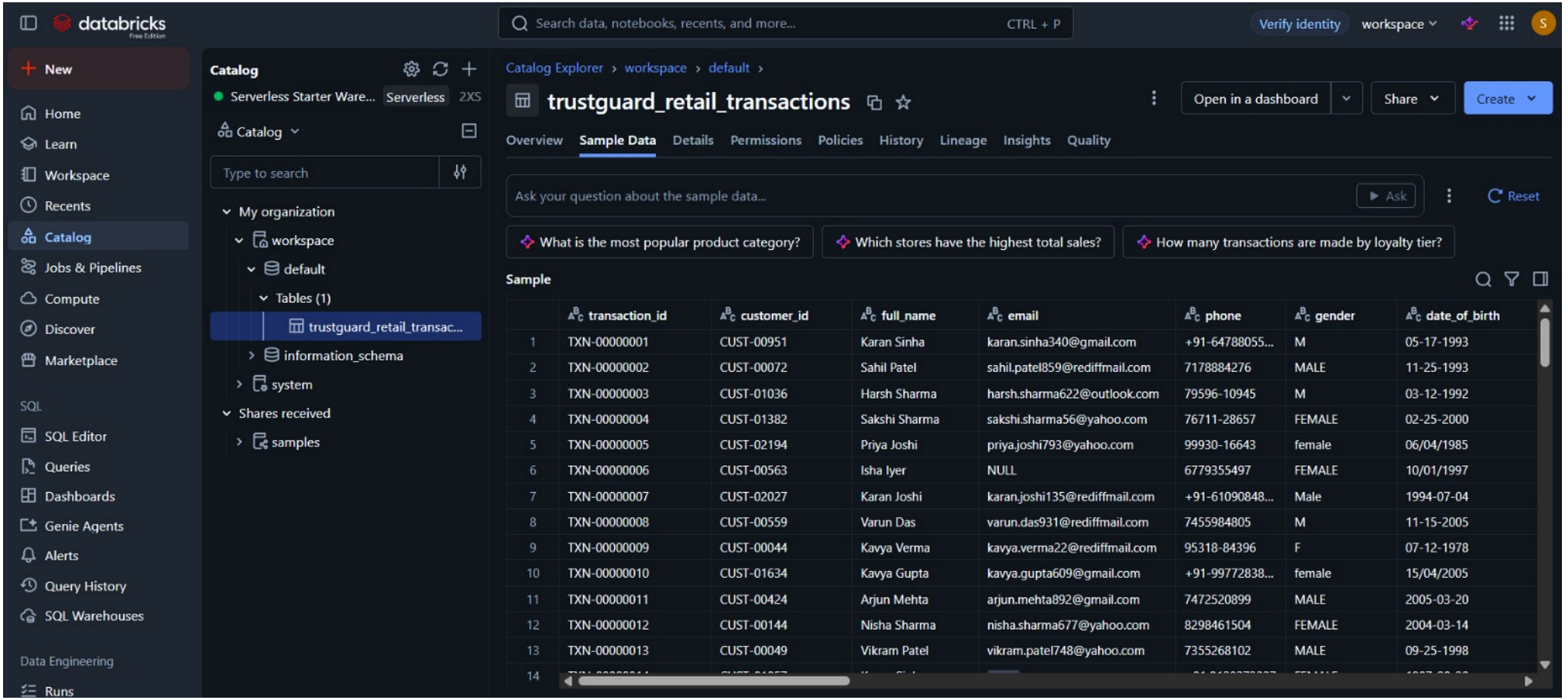
5. Important Output Tables

Table	Purpose
raw_transactions	Original uploaded data with extra metadata columns such as run_id, load_timestamp, and source_file_name.
dq_report	Shows data quality check results with passed records, failed records, and quality score.
rejected_records	Stores invalid records with clear rejection reasons.
clean_transactions	Stores cleaned transaction data after validation and deduplication.
clean_customers	Stores unique customer records derived from the cleaned transactions.
final_transactions	Main final analysis-ready transaction table.

Table	Purpose
customer_summary	Customer-level spend and order summary.
city_sales_report	City-wise and month-wise sales report.
anomaly_log	Records unusual transactions for review.
pipeline_log	Tracks pipeline run status and record counts.

6. Dataset Uploaded in Databricks

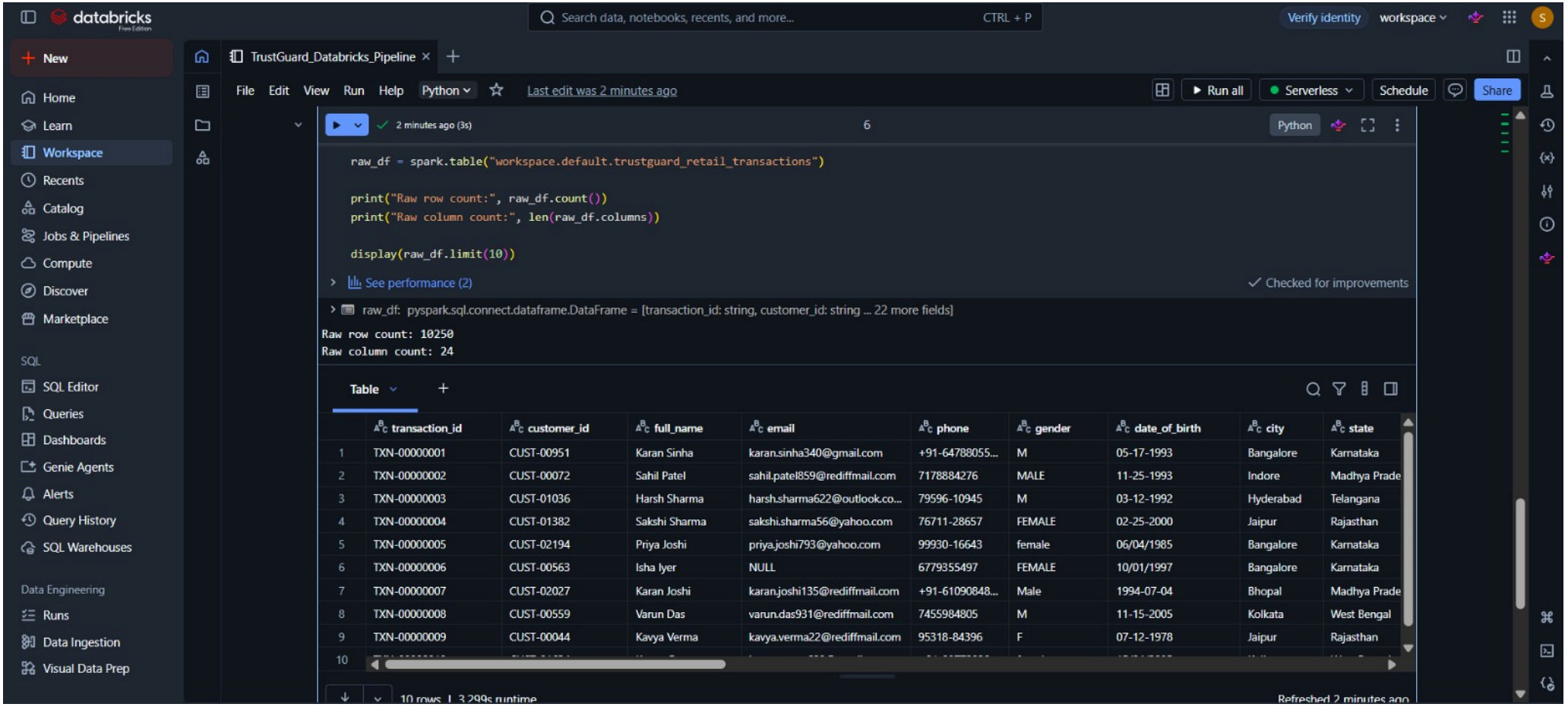
This screenshot shows the raw retail transaction dataset uploaded inside Databricks Catalog. The sample data confirms that the CSV was read correctly with columns such as transaction_id, customer_id, full_name, email, phone, gender, and date_of_birth.



	transaction_id	customer_id	full_name	email	phone	gender	date_of_birth
1	TXN-00000001	CUST-00951	Karan Sinha	karan.sinha340@gmail.com	+91-64788055...	M	05-17-1993
2	TXN-00000002	CUST-00072	Sahil Patel	sahil.patel859@rediffmail.com	7178884276	MALE	11-25-1993
3	TXN-00000003	CUST-01036	Harsh Sharma	harsh.sharma622@outlook.com	79596-10945	M	03-12-1992
4	TXN-00000004	CUST-01382	Sakshi Sharma	sakshi.sharma56@yahoo.com	76711-28657	FEMALE	02-25-2000
5	TXN-00000005	CUST-02194	Priya Joshi	priya.joshi793@yahoo.com	99930-16643	female	06/04/1985
6	TXN-00000006	CUST-00563	Isha Iyer	NULL	6779355497	FEMALE	10/01/1997
7	TXN-00000007	CUST-02027	Karan Joshi	karan.joshi135@rediffmail.com	+91-61090848...	Male	1994-07-04
8	TXN-00000008	CUST-00559	Varun Das	varun.das931@rediffmail.com	7455984805	M	11-15-2005
9	TXN-00000009	CUST-00044	Kavya Verma	kavya.verma22@rediffmail.com	95318-84396	F	07-12-1978
10	TXN-00000010	CUST-01634	Kavya Gupta	kavya.gupta609@gmail.com	+91-99772838...	female	15/04/2005
11	TXN-00000011	CUST-00424	Arjun Mehta	arjun.mehta892@gmail.com	7472520899	MALE	2005-03-20
12	TXN-00000012	CUST-00144	Nisha Sharma	nisha.sharma677@yahoo.com	8298461504	FEMALE	2004-03-14
13	TXN-00000013	CUST-00049	Vikram Patel	vikram.patel748@yahoo.com	7355268102	MALE	09-25-1998
14							

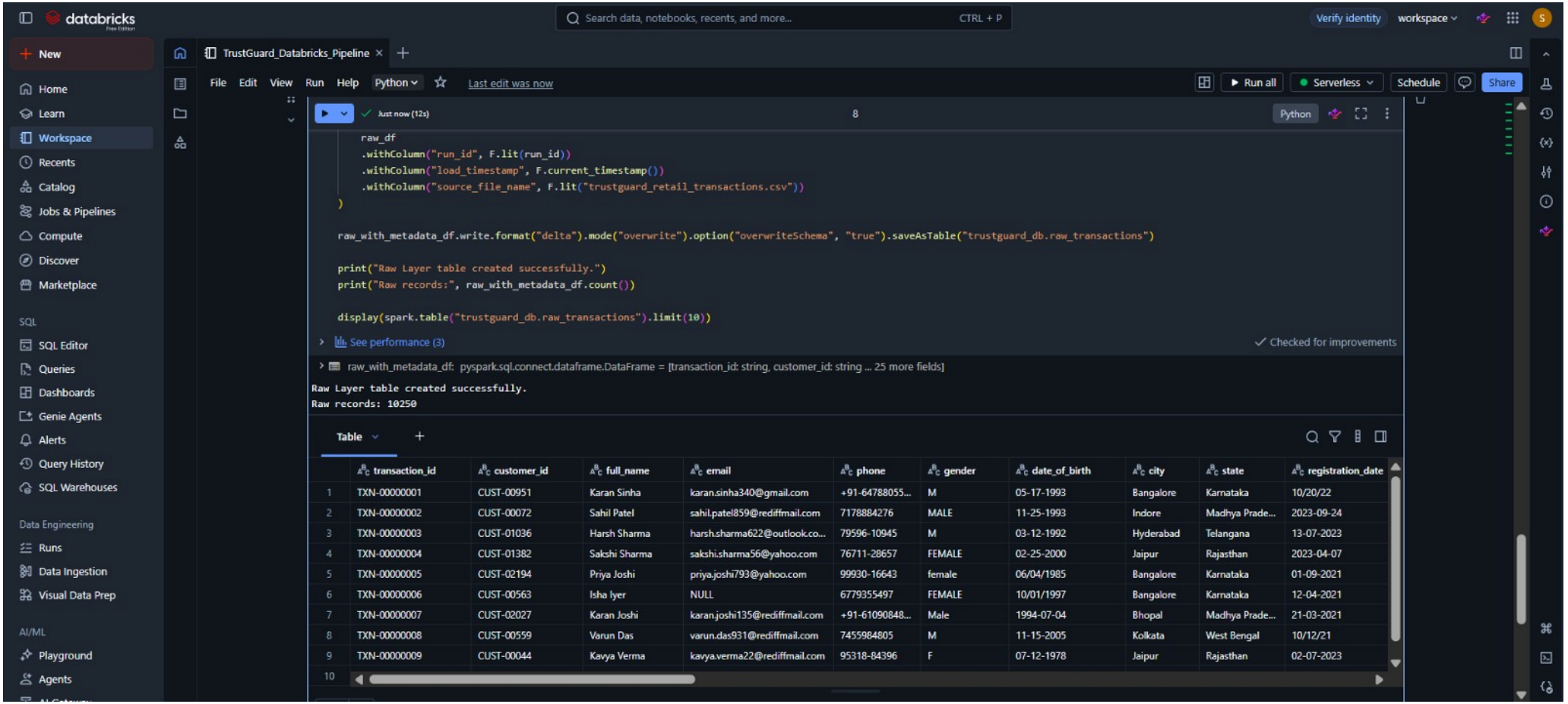
7. Raw Table Loaded in Notebook

This output confirms that the uploaded Databricks table was successfully loaded into a PySpark DataFrame. It shows 10,250 rows and 24 columns, so the notebook can now use the dataset for the pipeline.



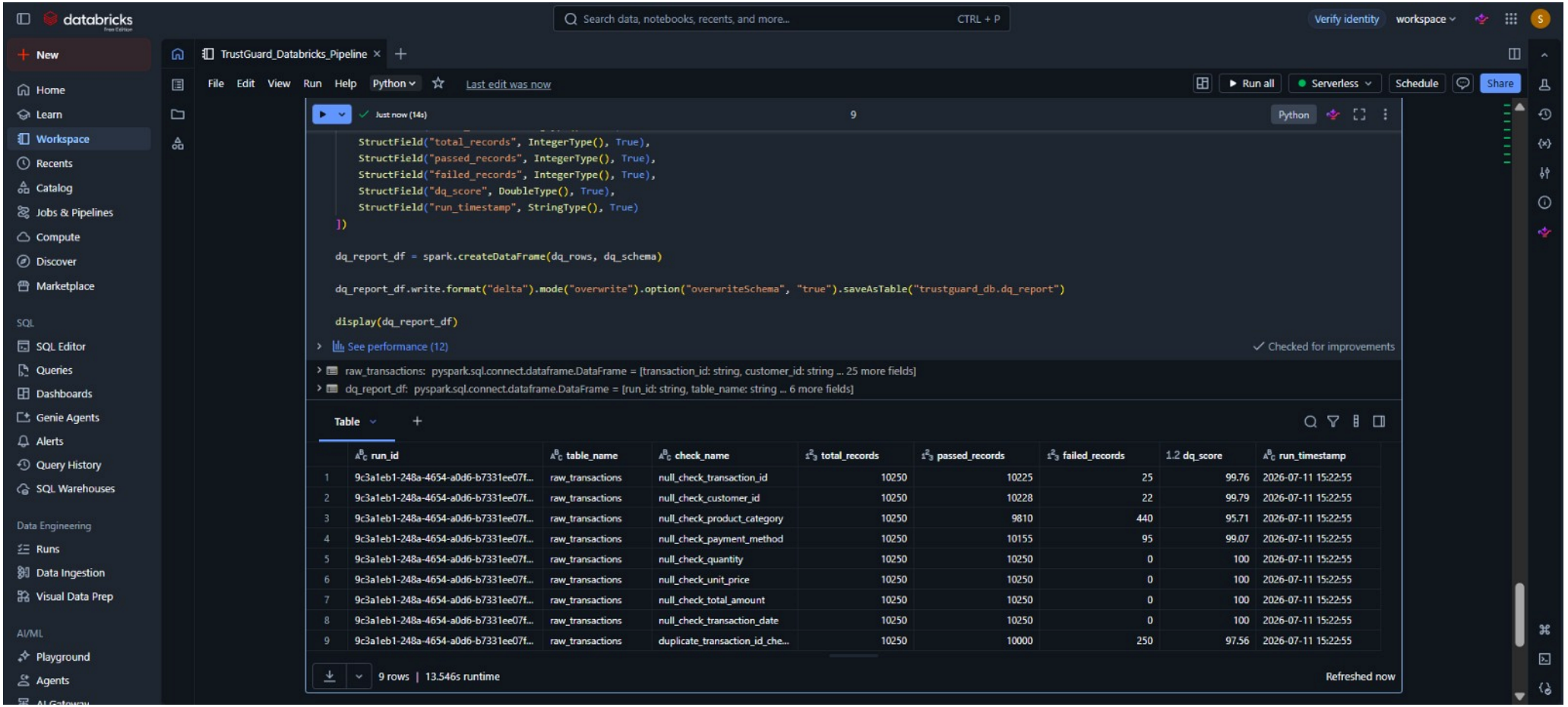
8. Raw Layer Delta Table

This screenshot shows the Raw Layer table created successfully. The raw data was stored with metadata columns such as run_id, load_timestamp, and source_file_name. No cleaning was applied in this layer.



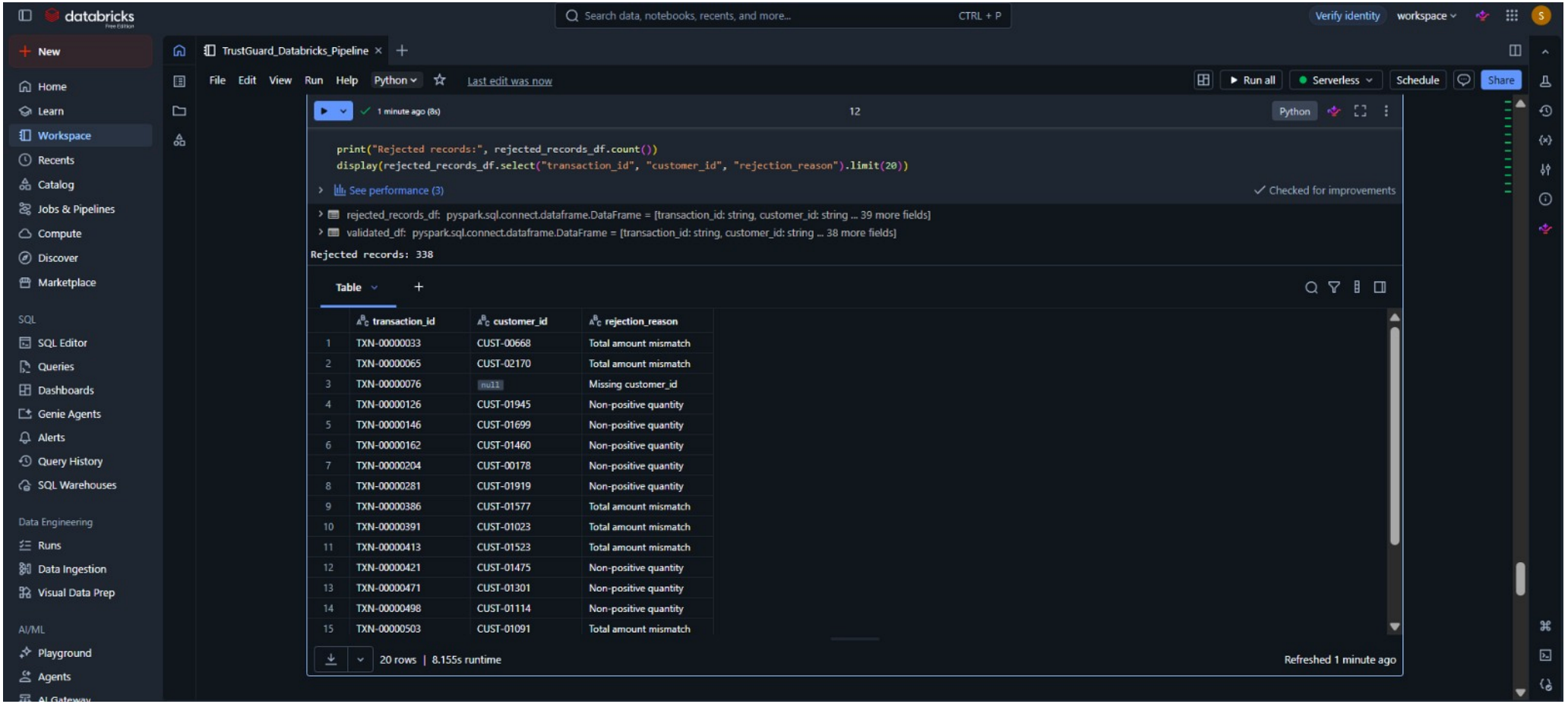
9. Initial Data Quality Report

This output shows the data quality report for the raw data. It counts failed records for missing values and duplicate transaction IDs. This helps identify how dirty the incoming data is before cleaning.



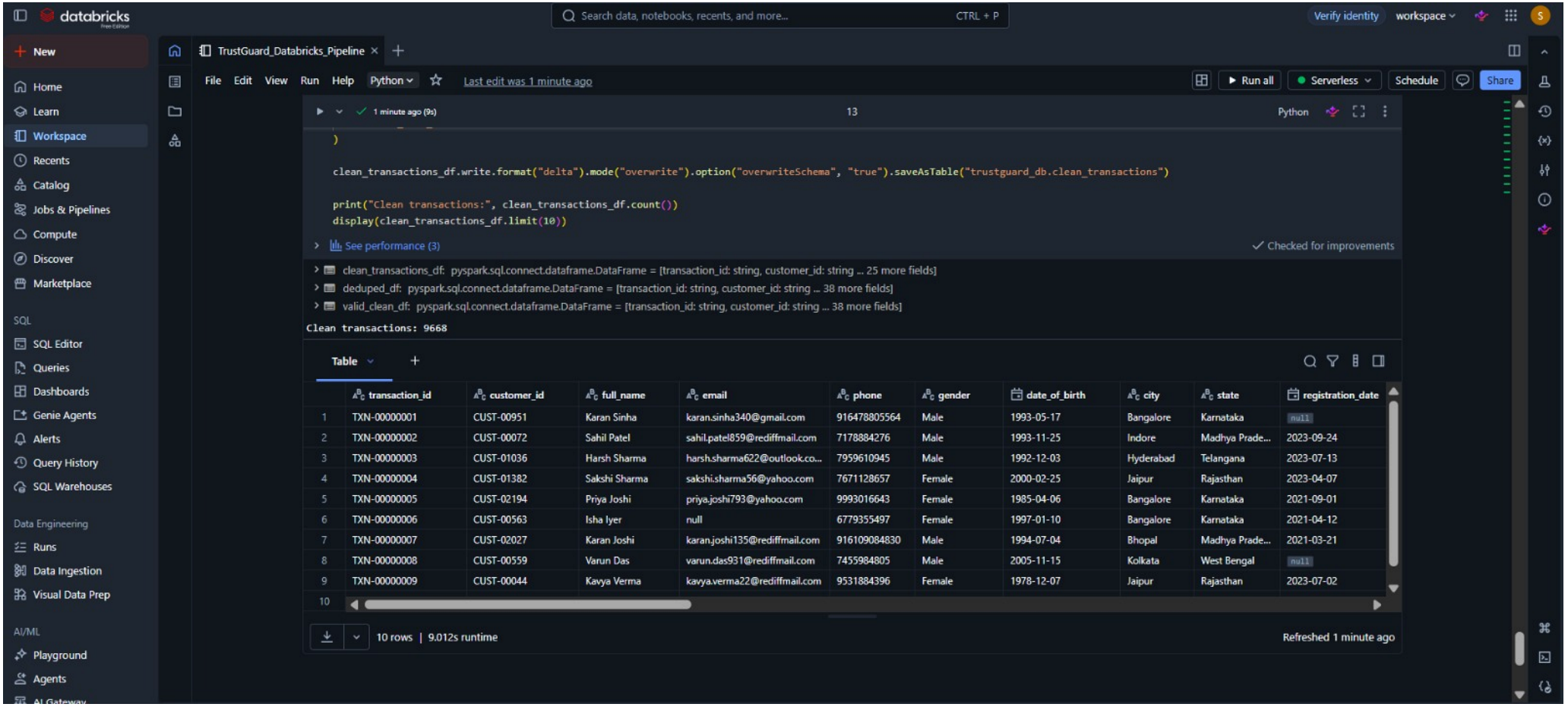
10. Rejected Records

This table contains records that failed critical checks. The rejection_reason column clearly explains why each record was rejected, such as total amount mismatch, missing customer_id, or non-positive quantity.



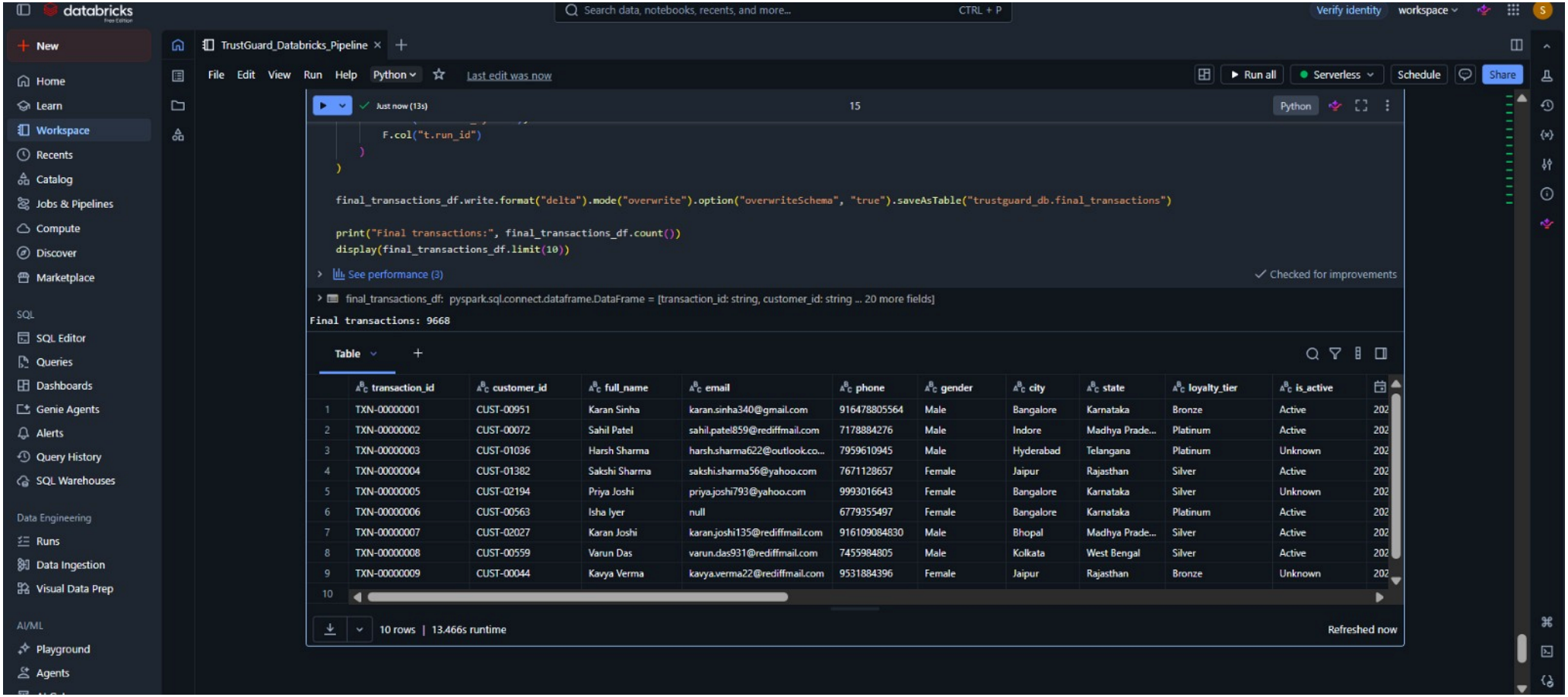
11. Clean Transactions Table

This screenshot shows the cleaned transactions after applying validation, standardization, deduplication, and type casting. The clean record count is lower than the raw count because invalid and duplicate records were handled.



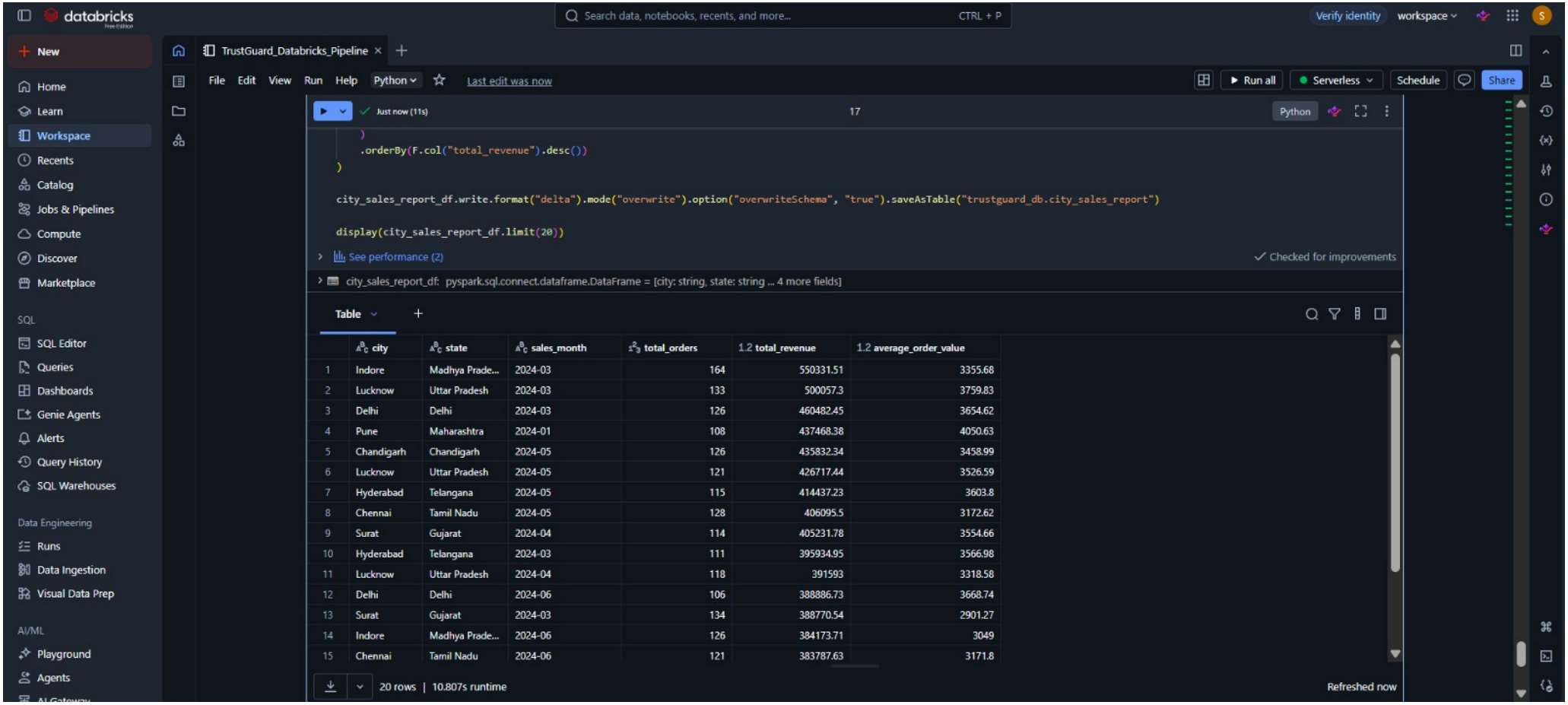
12. Final Transactions Table

This output shows the final analysis-ready transaction table. It contains clean transaction data joined with customer details and is used for business reporting.



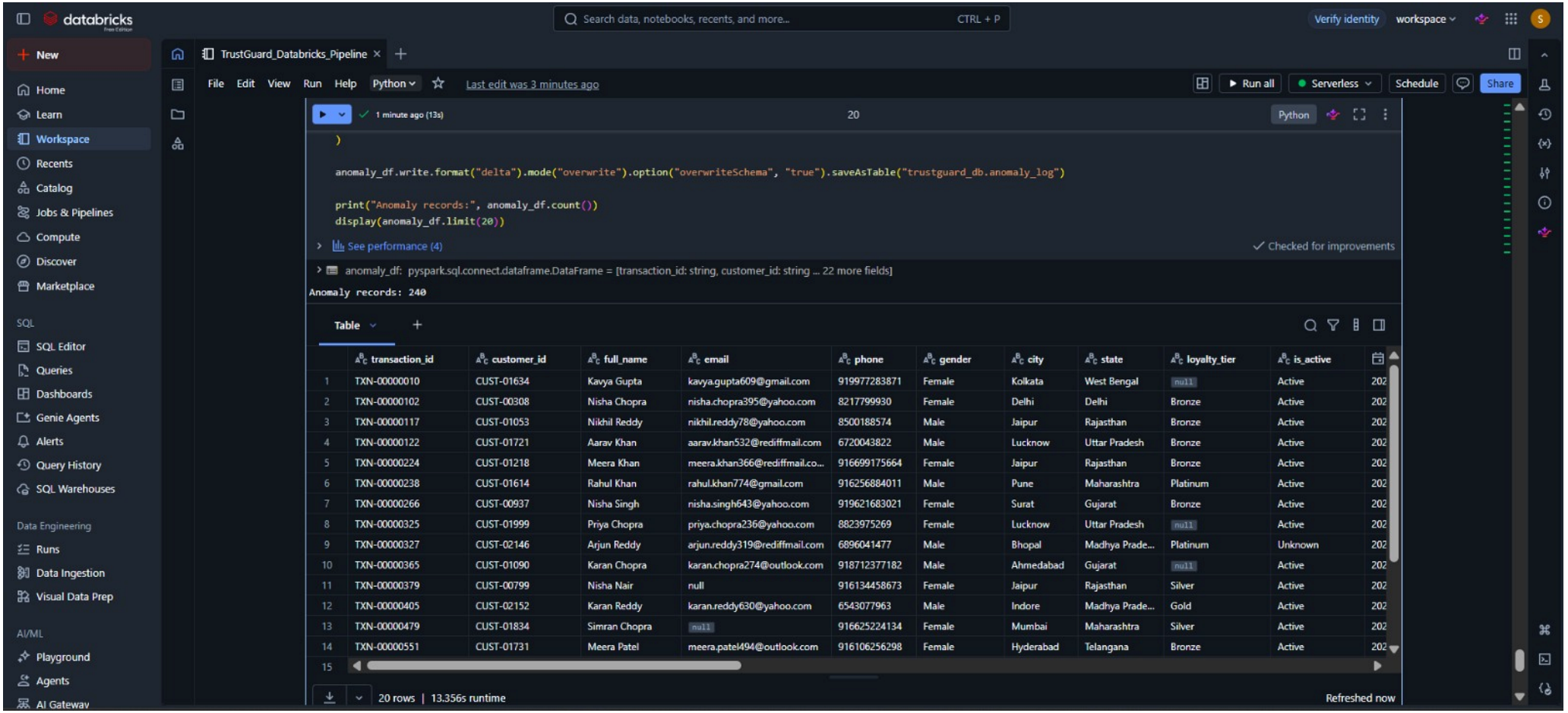
13. City Sales Report

This report shows city-wise and month-wise sales performance. It includes total orders, total revenue, and average order value, which are useful for business analysis.



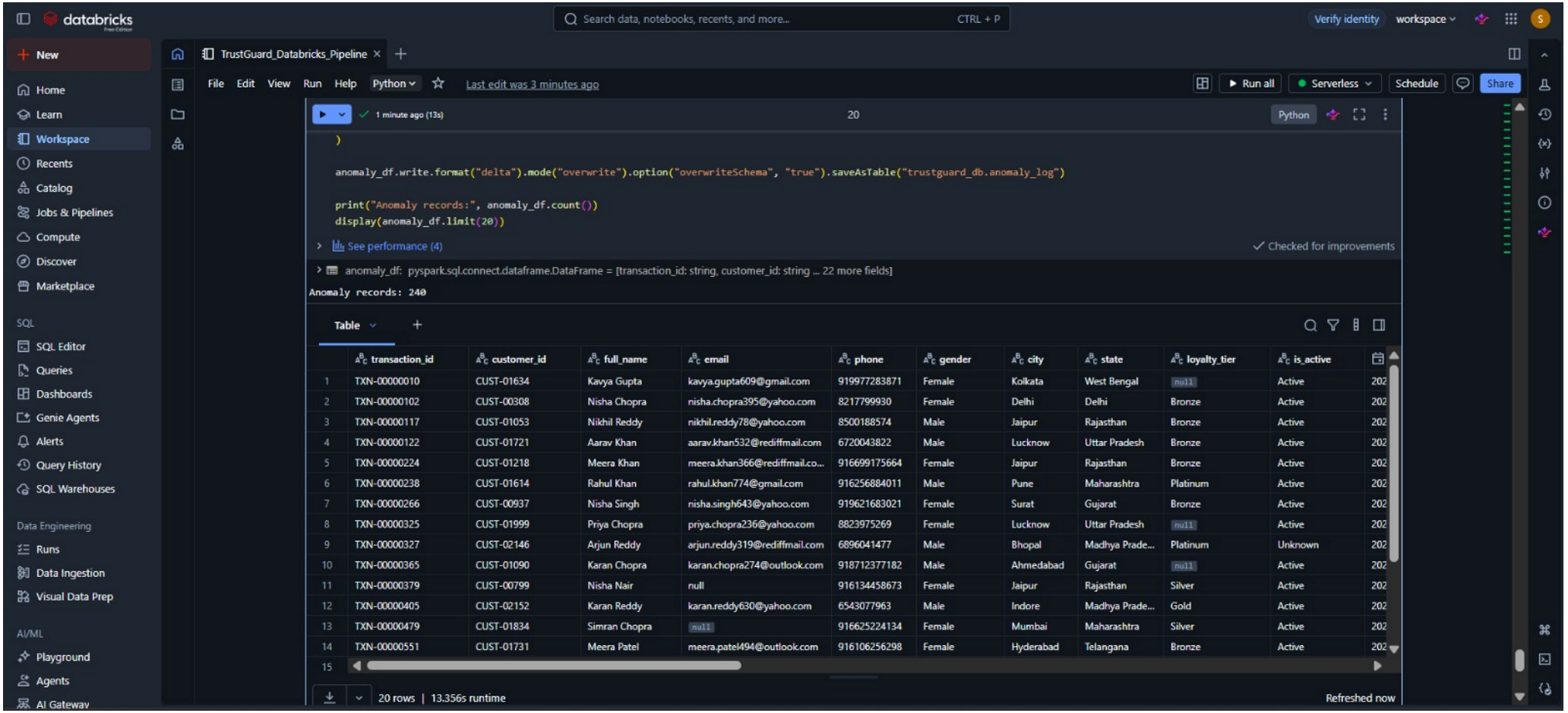
14. Anomaly Log

This screenshot shows unusual transactions flagged by the anomaly detection step. These records may have very high quantity or transaction amount and can be reviewed separately.



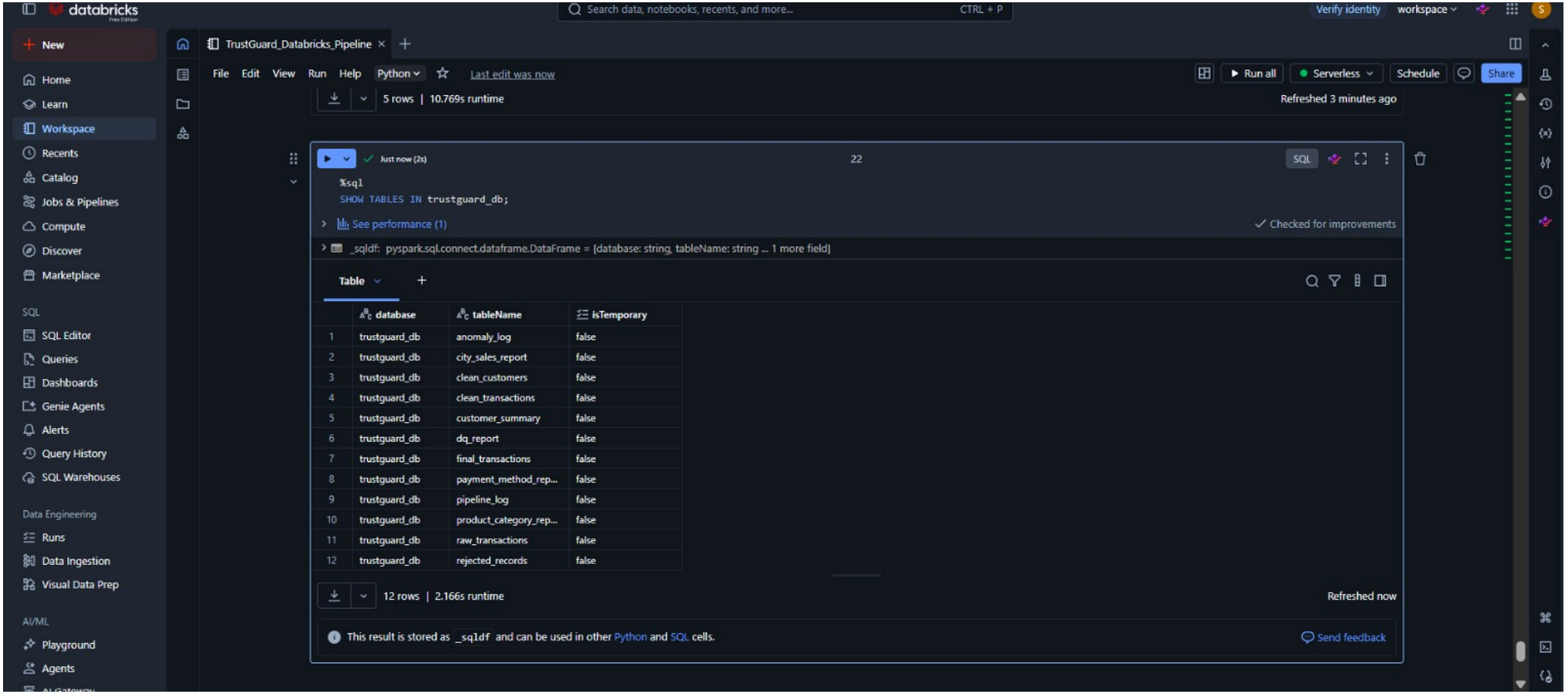
15. Pipeline Log

The pipeline log tracks each major pipeline step with record counts and status. It helps confirm that the pipeline ran successfully and makes the workflow easier to monitor.



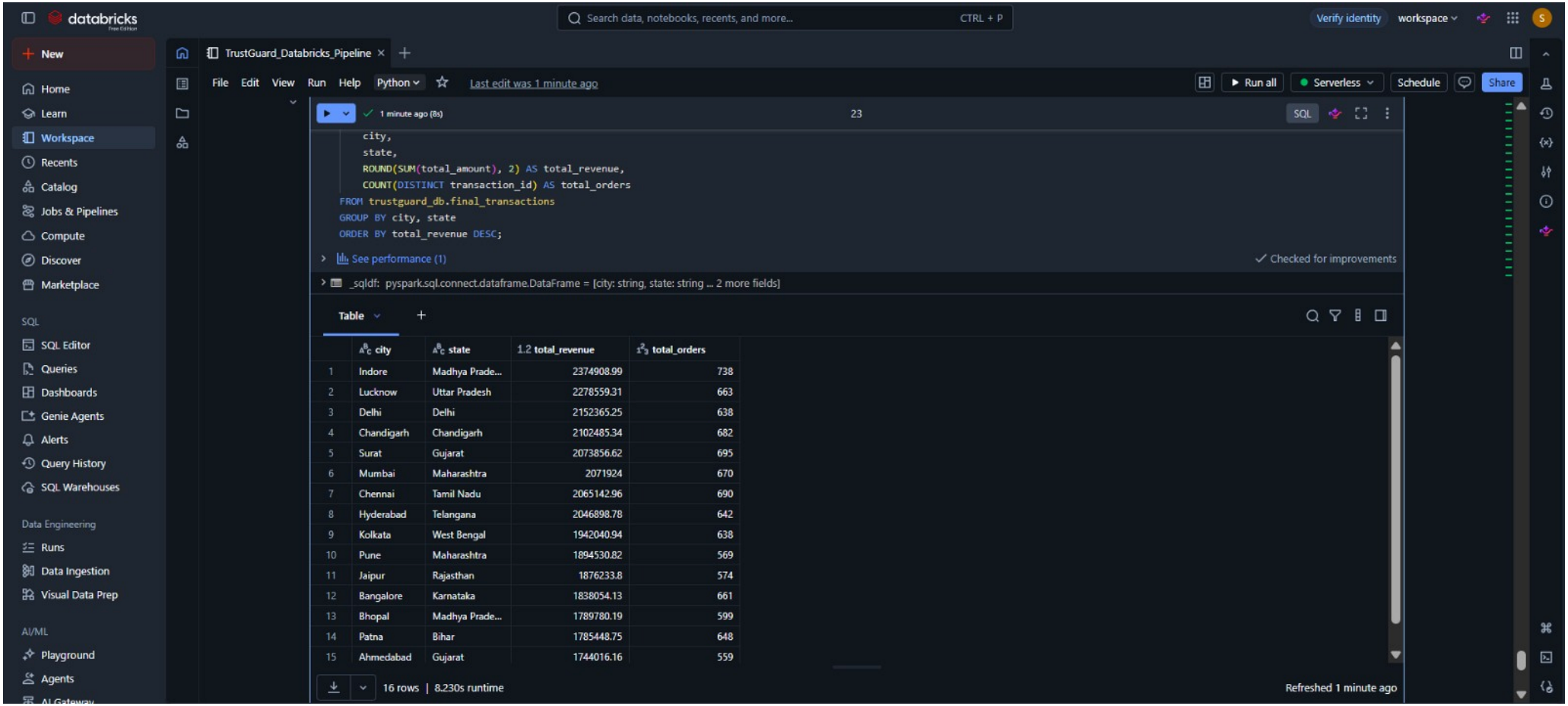
16. Tables Created in Databricks

This screenshot shows all tables created under the trustguard_db schema. It confirms that the Raw, Clean, Final, DQ, rejected, anomaly, and pipeline log tables were created successfully.



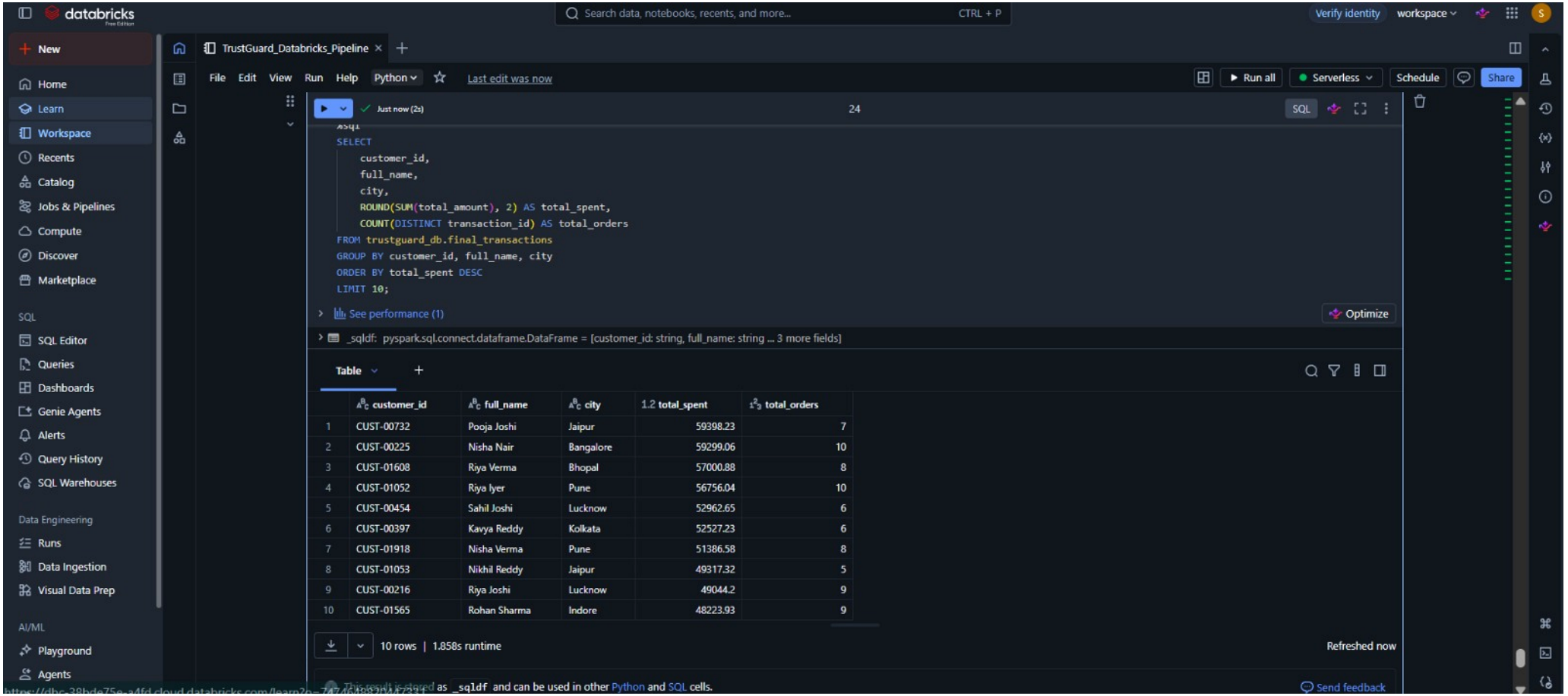
17. SQL Report - City Revenue

This SQL output shows total revenue and total orders by city and state. It proves that the final clean table can be queried for business reports.



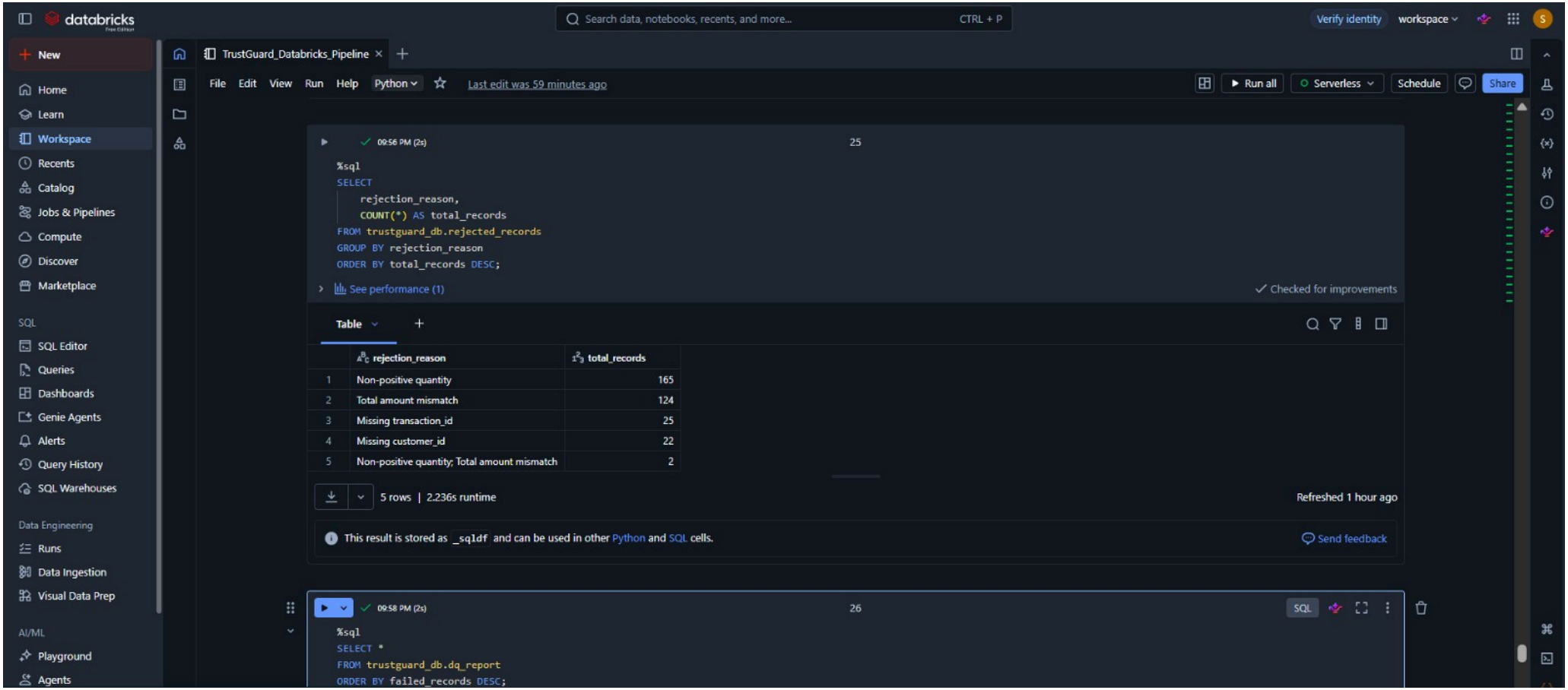
18. SQL Report - Top Customers

This report shows the top 10 customers by total spend. It uses the final_transactions table and helps identify high-value customers.



19. SQL Report - Rejected Records Summary

This SQL output groups rejected records by rejection reason. It helps understand the most common data quality problems in the raw dataset.



20. SQL Report - Data Quality Summary

This output shows the data quality checks ordered by failed records. It gives a clear view of which checks had the highest number of failures.